

Nama : Zibran Rizki Arta

NIM : 224308049

Kelas : TKA-6B

Mata Kuliah Praktikum Kontrol Cerdas

Tugas Kontrol PID

Dalam proses pengerjaan tugas kontrol PID pada mata kuliah Praktikum Kontrol Cerdas menggunakan *software* Matlab dan Arduino IDE, ada beberapa tahapan dalam pengerjaannya diantaranya, sebagai berikut :

1. Kode Program *Transfer Fuction* :

```
clear
clc
% Parameter Motor
R = 0.4;
L = 2.7;
J = 0.0004;
B = 0.0022;
Ke = 0.015;
Kt = 0.05;

% Numerator dan Denominator
num = Ke;
den = [L*J (L*B + R*J) (R*B + Ke*Kt)];

% Transfer Function (Open-loop)
Gs = tf(num, den);

% PID parameters
kp = 53.7208;
Ti = 0.73245;
Td = 0.17711;
ki = kp / Ti;
kd = kp * Td;

% Buat controller PID
PID = pid(kp, ki, kd);

% Closed-loop transfer function (Negative feedback, unity)
T = feedback(PID * Gs, 1);

% Step response
step(T)
title('Step Response with PID Controller')
grid on
```

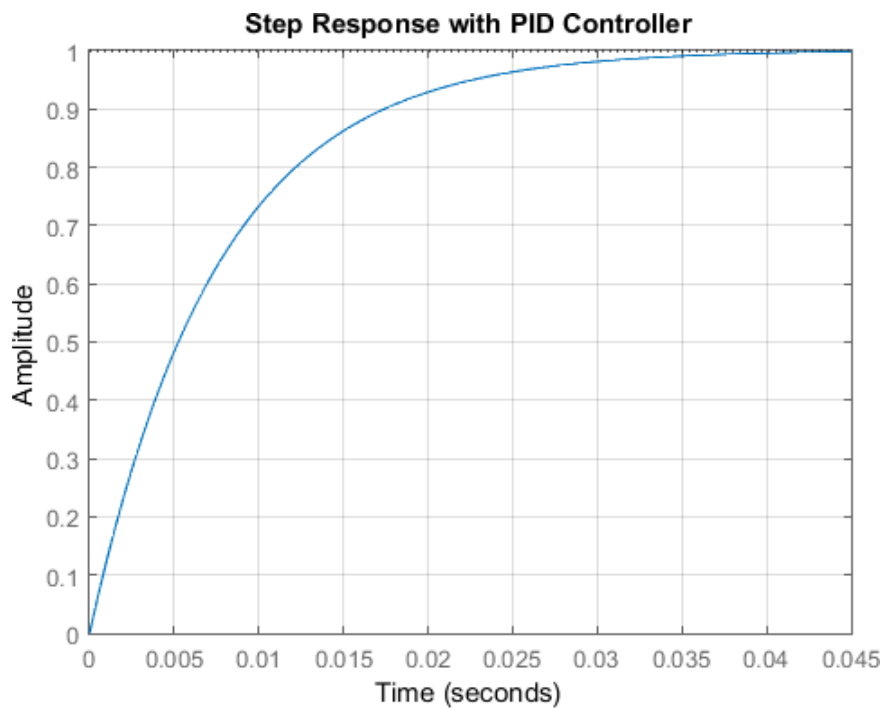
```

% Hitung karakteristik step response
info = stepinfo(T)

% Tampilkan hasil karakteristik
fprintf('\nKarakteristik Step Response:\n');
fprintf('Rise Time    : %.4f s\n', info.RiseTime);
fprintf('Peak Time     : %.4f s\n', info.PeakTime);
fprintf('Overshoot      : %.2f %%\n', info.Overshoot);
fprintf('Settling Time  : %.4f s\n', info.SettlingTime);

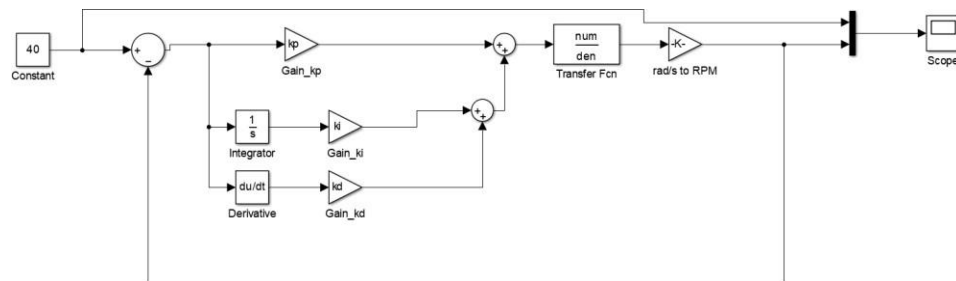
```

2. Output Hasil Running Program :



Info =	Karakteristik Step Response:
Rise Time: 0.0166	Rise Time : 0.0166 s
Settling Time: 0.0294	Peak Time : 0.0554 s
Settling Min: 0.9002	Overshoot : 0.10 %
Settling Max: 1.0010	Settling Time : 0.0294 s
Overshoot: 0.1002	

3. Model Simulink :



Hasil Simulink :



4. Kode Program Arduino IDE :

```
#include <WiFi.h>
#include <PubSubClient.h>
// =====
// KONFIGURASI WIFI & MQTT
// =====
const char* ssid = "TKA 6Barokah";
const char* password = "65432100";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
WiFiClient espClient;
PubSubClient client(espClient);
// =====
// PIN KONFIGURASI
// =====
const int pinMotorPWM = 5;
const int pinREn = 21;
```

```

const int pinLEn = 22;
const int encoderPinA = 34;
const int encoderPinB = 35;
// =====
// VARIABEL PID & MOTOR
// =====
kp = 53.7208;
Ti = 0.73245;
Td = 0.17711;
ki = kp / Ti;
kd = kp * Td;
float error = 0, lastError = 0, integral = 0;
float targetSpeedKmh = 0;
float actualSpeedKmh = 0;
int pidPWM = 0;
int throttleLevel = 0;
String motorDirection = "forward"; // Default arah
// =====
// ENCODER
// =====
volatile long encoderCount = 0;
unsigned long lastSpeedCheck = 0;
void IRAM_ATTR encoderISR() {
  encoderCount++;
}
// =====
// WIFI DAN MQTT SETUP
// =====
void setup_wifi() {
  Serial.print("Connecting to WiFi ");
  Serial.print(ssid);
  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.println("WiFi connected, IP: " + WiFi.localIP().toString());
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connecting to MQTT...");
    String clientId = "ESP32Client-";
    clientId += String(random(0xffff), HEX);
    if (client.connect(clientId.c_str())) { // Connect tanpa user/password
      Serial.println("connected!");
    }
  }
}

```

```

        client.subscribe("motor/throttle");
        client.subscribe("motor/direction");
    } else {
        Serial.print("failed, rc=");
        Serial.print(client.state());
        Serial.println(" try again in 5 seconds");
        delay(5000);
    }
}
}
// =====
// CALLBACK MQTT
// =====
void callback(char* topic, byte* payload, unsigned int length) {
    String msg;
    for (unsigned int i = 0; i < length; i++) {
        msg += (char)payload[i];
    }

    Serial.print("[MQTT] Topic: ");
    Serial.print(topic);
    Serial.print(" | Message: ");
    Serial.println(msg);

    if (String(topic) == "motor/throttle") {
        int throttle = msg.toInt();
        throttleLevel = constrain(throttle, 0, 3);
        switch (throttleLevel) {
            case 1: targetSpeedKmh = 5; break;
            case 2: targetSpeedKmh = 10; break;
            case 3: targetSpeedKmh = 15; break;
        }
        Serial.println("[MQTT] Throttle level set: " + String(throttleLevel));
    }

    if (String(topic) == "motor/direction") {
        msg.toLowerCase();
        if (msg == "forward" || msg == "reverse" || msg == "neutral") {
            motorDirection = msg;
            Serial.println("[MQTT] Direction set: " + motorDirection);
        }
    }
}

// =====
// SETUP
// =====
void setup() {

```

```

Serial.begin(115200);

pinMode(pinREn, OUTPUT);
pinMode(pinLEn, OUTPUT);

ledcSetup(0, 5000, 8); // 5kHz, 8-bit PWM
ledcAttachPin(pinMotorPWM, 0);

pinMode(encoderPinA, INPUT_PULLUP);
pinMode(encoderPinB, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(encoderPinA), encoderISR,
RISING);

setup_wifi();
client.setServer(mqtt_server, mqtt_port);
client.setCallback(callback);
}

// =====
// MENGHITUNG KECEPATAN
// =====
void updateSpeed() {
  noInterrupts();
  long count = encoderCount;
  encoderCount = 0;
  interrupts();

  float rpm = (count / 20.0) * 60.0; // 20 pulses per rotation
  float wheelCircumference = 3.14 * 0.1; // Diameter roda 10 cm
  actualSpeedKmh = (rpm * wheelCircumference * 60.0) / 1000.0;

  Serial.println("[ESP32] Actual Speed: " + String(actualSpeedKmh, 2) + "
km/h");
  client.publish("motor/actual_speed", String(actualSpeedKmh, 2).c_str());
}

// =====
// KONTROL ARAH
// =====
void controlDirection() {
  if (motorDirection == "forward") {
    digitalWrite(pinREn, HIGH);
    digitalWrite(pinLEn, LOW);
  } else if (motorDirection == "reverse") {
    digitalWrite(pinREn, LOW);
    digitalWrite(pinLEn, HIGH);
  } else { // neutral
    digitalWrite(pinREn, LOW);

```

```

        digitalWrite(pinLEn, LOW);
    }
}

// =====
// PID CONTROL
// =====
void PIDControl() {
    if (motorDirection == "neutral" || targetSpeedKmh == 0) {
        ledcWrite(0, 0);
        pidPWM = 0;
        integral = 0;
        lastError = 0;
        return;
    }

    error = targetSpeedKmh - actualSpeedKmh;
    integral += error;
    float derivative = error - lastError;
    lastError = error;

    float output = Kp * error + Ki * integral + Kd * derivative;
    pidPWM = constrain((int)output, 0, 255);
    ledcWrite(0, pidPWM);

    Serial.println("[ESP32] PID PWM: " + String(pidPWM));
    client.publish("motor/pid_output", String(pidPWM).c_str());
}

// =====
// LOOP UTAMA
// =====
void loop() {
    if (!client.connected()) {
        reconnect();
    }
    client.loop();

    controlDirection();

    unsigned long now = millis();
    if (now - lastSpeedCheck >= 1000) {
        lastSpeedCheck = now;
        updateSpeed();
        PIDControl();
    }
}

```

ANALISIS

Sistem kontrol Proporsional-Integral-Derivatif (PID) telah terbukti menjadi metode yang sangat efektif dalam pengaturan kecepatan motor DC, menawarkan stabilitas dan respons dinamis yang optimal. Proyek ini secara komprehensif menguraikan perancangan dan implementasi kontrol PID, memanfaatkan keunggulan simulasi perangkat lunak sekaligus realisasi fisik. Tahap awal melibatkan pemodelan sistem motor DC yang presisi, mempertimbangkan parameter krusial seperti resistansi (R), induktansi (L), momen inersia (J), koefisien gesekan (B), konstanta tegangan balik (K_e), dan konstanta torsi (K_t). Dengan pemodelan ini, fungsi transfer *open-loop* motor DC berhasil disusun, menjadi dasar bagi desain kontroler PID. Proses penyesuaian nilai *gain* kontroler yaitu proporsional (K_p), integral (K_i), dan derivatif (K_d) dilakukan secara cermat melalui *tuning*, memastikan kinerja sistem yang diharapkan.

Hasil simulasi yang dilakukan menggunakan MATLAB secara jelas menunjukkan kapabilitas kontrol PID yang dirancang dalam menghasilkan respons sistem yang cepat dan sangat stabil. Indikator kinerja yang terukur membuktikan hal ini: nilai rise time yang sangat rendah (0,0166 detik) menunjukkan kecepatan sistem dalam mencapai setpoint; settling time (0,0294 detik) mengindikasikan waktu yang dibutuhkan sistem untuk stabil; peak time (0,0554 detik) menunjukkan waktu puncak respons; dan overshoot yang minimal (0,10%) menggarisbawahi akurasi dan stabilitas tanpa osilasi berlebihan. Angka-angka ini secara kolektif menegaskan bahwa sistem kontrol PID ini mampu memberikan waktu tanggap yang luar biasa cepat dengan deviasi yang hampir tidak signifikan dari nilai referensi yang diinginkan, mencerminkan kinerja kendali yang benar-benar optimal.

Transisi dari simulasi ke implementasi fisik direalisasikan melalui penggunaan mikrokontroler ESP32, yang dikonfigurasi untuk beroperasi dalam lingkungan jaringan. Perintah kontrol diterima melalui antarmuka MQTT yang berbasis Wi-Fi, memungkinkan komunikasi nirkabel yang fleksibel. Kode program yang dikembangkan dalam Arduino IDE secara eksplisit menunjukkan bagaimana kontroler PID diterapkan untuk mengatur kecepatan motor. Pengaturan ini didasarkan pada pembacaan *real-time* dari sensor *rotary encoder*, yang menyediakan umpan balik kecepatan aktual motor. Sistem kemudian menghitung *error* antara kecepatan aktual dan kecepatan target, dan berdasarkan perhitungan PID, sinyal PWM disesuaikan untuk mengendalikan motor. Lebih jauh, sistem ini tidak hanya mengendalikan kecepatan tetapi juga memungkinkan pengaturan arah putaran motor maju (*forward*), mundur (*reverse*), dan netral (*neutral*) melalui

perintah MQTT, memberikan kontrol arah gerakan yang komprehensif dan adaptif.

Keselarasan antara desain teoritis dan aplikasi praktis merupakan salah satu poin paling krusial dari proyek ini. Integrasi yang mulus antara hasil simulasi dari software MATLAB dengan implementasi pada mikrokontroler ESP32 memperlihatkan konsistensi yang tinggi. Ini membuktikan bahwa efektivitas kontrol PID tidak hanya terbatas pada lingkungan simulasi ideal, tetapi juga secara konsisten terwujud dalam kondisi operasional nyata. Pendekatan holistik ini, yang mencakup pemodelan, simulasi, dan implementasi fisik, menjadikan sistem kontrol PID ini sangat relevan dan menjanjikan untuk berbagai aplikasi dalam sistem otomasi dan robotika yang bergantung pada motor DC. Potensi penerapannya sangat luas, mulai dari kendaraan otonom mini hingga sistem manufaktur otomatis, di mana kontrol kecepatan yang presisi adalah kunci keberhasilan.